

Algorithm 837: AMD, An Approximate Minimum Degree Ordering Algorithm

PATRICK R. AMESTOY, ENSEEIHT-IRIT, and TIMOTHY A. DAVIS

University of Florida

and

IAIN S. DUFF

CERFACS and Rutherford Appleton Laboratory

AMD is a set of routines that implements the approximate minimum degree ordering algorithm to permute sparse matrices prior to numerical factorization. There are versions written in both C and Fortran 77. A MATLAB interface is included.

Categories and Subject Descriptors: G.1.3 [**Numerical Analysis**]: Numerical Linear Algebra—*Linear systems (direct and iterative methods); sparse structural, and very large systems (direct and iterative methods)*; G.4 [**Mathematics of Computing**]: Mathematical Software—*Algorithm design and analysis; efficiency*

General Terms: Algorithms, Experimentation, Performance

Additional Key Words and Phrases: Linear equations, minimum degree, ordering methods, sparse matrices

1. OVERVIEW

AMD is a set of routines for reordering a sparse matrix prior to numerical factorization. It uses an approximate minimum degree ordering algorithm [Amestoy et al. 1996] to find a permutation matrix $\mathbf{P}$ so that the Cholesky factorization $\mathbf{PAP}^T = \mathbf{LL}^T$ has fewer (often much fewer) nonzero entries than

The work of T. A. Davis was supported by the National Science Foundation, under grants ASC-9111263, DMS-9223088, and CCR-0203270.

Portions of the work of T. A. Davis were done while on sabbatical at Stanford University and Lawrence Berkeley National Laboratory (with funding from Stanford University and the SciDAC program).

The work of I. S. Duff was supported by the EPSRC under grant GR/R46441.

Authors' addresses: P. R. Amestoy, ENSEEIHT-IRIT, 2 rue Camichel 31017 Toulouse, France, Web: <http://www.enseeiht.fr/~amestoy>; T. A. Davis, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, FL, Web: <http://www.cise.ufl.edu/~davis>; I. S. Duff, Rutherford Appleton Laboratory, Chilton, Didcot, Oxon OX11 0QX, England; email: i.s.duff@rl.ac.uk, Web: <http://www.numerical.rl.ac.uk/people/isd/isd.html>.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or direct commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 1515 Broadway, New York, NY 10036 USA, fax: +1 (212) 869-0481, or permissions@acm.org.

© 2004 ACM 0098-3500/04/0900-0381 \$5.00

the Cholesky factorization of $\mathbf{A}$. The algorithm is typically much faster than other ordering methods and minimum degree ordering algorithms that compute an exact degree [George and Liu 1989]. Some methods, such as approximate deficiency [Rothberg and Eisenstat 1998] and graph-partitioning based methods [Hendrickson and Rothberg 1999; Karypis and Kumar 1998; Pellegrini et al. 2000; Schulze 2001] can produce better orderings, depending on the matrix.

The algorithm starts with an undirected graph representation of a symmetric sparse matrix $\mathbf{A}$. Node i in the graph corresponds to row and column i of the matrix, and there is an edge (i, j) in the graph if a_{ij} is nonzero. The degree of a node is initialized to the number of off-diagonal nonzeros in row i , which is the size of the set of nodes adjacent to i in the graph.

The selection of a pivot a_{ii} from the diagonal of $\mathbf{A}$ and the first step of Gaussian elimination corresponds to one step of graph elimination. Numerical fill-in causes new nonzero entries in the matrix (fill-in refers to nonzeros in $\mathbf{L}$ that are not in $\mathbf{A}$). Node i is eliminated and edges are added to its neighbors so that they form a clique (or *element*). To reduce fill-in, node i is selected as the node of least degree in the graph. This process repeats until the graph is eliminated.

The clique is represented implicitly. Rather than listing all the new edges in the graph, a single list of nodes is kept which represents the clique. This list corresponds to the nonzero pattern of the first column of $\mathbf{L}$. As the elimination proceeds, some of these cliques become subsets of subsequent cliques, and are removed. This graph can be stored in place, that is using the same amount of memory as the original graph.

The most costly part of the minimum degree algorithm is the recomputation of the degrees of nodes adjacent to the current pivot element. Rather than keep track of the exact degree, the approximate minimum degree algorithm finds an upper bound on the degree that is easier to compute. For nodes of least degree, this bound tends to be tight. Using the approximate degree instead of the exact degree leads to a substantial savings in run time, particularly for very irregularly structured matrices. It has no effect on the quality of the ordering.

In the C version of AMD, the elimination phase is followed by an elimination tree post-ordering. This has no effect on fill-in, but reorganizes the ordering so that the subsequent numerical factorization is more efficient. It also includes a pre-processing phase in which nodes of very high degree are removed (without causing fill-in), and placed last in the permutation $\mathbf{P}$. This reduces the run time substantially if the matrix has a few rows with many nonzero entries, and has little effect on the quality of the ordering. The C version operates on the symmetric nonzero pattern of $\mathbf{A} + \mathbf{A}^T$, so it can be given an unsymmetric matrix, or either the lower or upper triangular part of a symmetric matrix.

The two Fortran versions of AMD are essentially identical to two versions of the AMD algorithm discussed in an earlier article [Amestoy et al. 1996] (approximate minimum external degree, both with and without aggressive absorption). For a discussion of the long history of the minimum degree algorithm, see George and Liu [1989].

2. USING AMD IN MATLAB

The MATLAB statement `p=amd(A)` finds a permutation vector `p` such that the Cholesky factorization `chol(A(p,p))` is typically sparser than `chol(A)`. If `A` is unsymmetric, `amd(A)` is identical to `amd(A+A')` (ignoring numerical cancellation). The AMD mexFunction is much faster than the built-in MATLAB symmetric minimum degree ordering methods, SYMAMD and SYMMMD. Its ordering quality is comparable to SYMAMD, and better than SYMMMD [Davis et al. 2004].

An optional input argument can be used to modify the control parameters for AMD (aggressive absorption, dense row/column handling, and printing of statistics). An optional output argument provides statistics on the ordering, including an analysis of the fill-in and the floating-point operation count for a subsequent factorization. For more details (once AMD is installed), type `help amd` in the MATLAB command window.

3. USING AMD IN A C PROGRAM

The C-callable AMD library consists of five user-callable routines and one include file. There are two versions of each of the routines, with `int` and `long` integers. Only the `int` versions are described here; the `long` versions are analogous.

- (1) `amd_order`. Computes the approximate minimum degree ordering of an n -by- n matrix **A**. Returns a permutation vector `P` of size n , where `P[k] = i` if row and column `i` are the `k`th row and column in the permuted matrix. This routine allocates its own memory of size $1.2e + 9n$ integers, where e is the number of nonzeros in $\mathbf{A} + \mathbf{A}^T$. It computes statistics about the matrix **A**, such as the symmetry of its nonzero pattern, the number of nonzeros in **L**, and the number of floating-point operations required for Cholesky and LU factorizations (which are returned in the `Info` array). The user's input matrix is not modified. It returns `AMD_OK` if successful, `AMD_INVALID` if the matrix is invalid, or `AMD_OUT_OF_MEMORY` if out of memory.

```
#include "amd.h"
int n, Ap [n + 1], Ai [nz], P [n] ;
double Control [AMD_CONTROL], Info [AMD_INFO] ;
int result = amd_order (n, Ap, Ai, P, Control, Info) ;
```

- (2) `amd_defaults`. Sets the default control parameters in the `Control` array. These can then be modified as desired before passing the array to the other AMD routines.

```
#include "amd.h"
double Control [AMD_CONTROL] ;
amd_defaults (Control) ;
```

- (3) `amd_control`. Prints a description of the control parameters, and their values.

```
#include "amd.h"
double Control [AMD_CONTROL] ;
amd_control (Control) ;
```

- (4) `amd_info`. Prints a description of the statistics computed by AMD, and their values.

```
#include "amd.h"
double Info [AMD_INFO] ;
amd_info (Info) ;
```

- (5) `amd_preprocess`. Removes duplicate entries and sorts each column of its input $\mathbf{A}$, and returns the nonzero pattern of the transpose, $\mathbf{R} = \mathbf{A}^T$. It returns the transpose because this is the simplest way to sort a matrix and remove duplicate entries. Either $\mathbf{A}$ or $\mathbf{A}^T$ can be passed to `amd_order` with little effect on the ordering (except for minor tie-breaking changes).

```
#include "amd.h"
int n, Ap [n + 1], Ai [nz], Rp [n + 1], Ri [nz] ;
int result = amd_preprocess (n, Ap, Ai, Rp, Ri) ;
```

The nonzero pattern of the matrix $\mathbf{A}$ is represented in compressed column form. For an n -by- n matrix $\mathbf{A}$ with nz nonzero entries, the representation consists of two arrays: `Ap` of size $n + 1$ and `Ai` of size nz . The row indices of entries in column j are stored in `Ai[Ap[j] .. Ap[j + 1] - 1]`. For `amd_order`, no duplicate row indices may be present, and the row indices in any given column must be sorted in ascending order. The matrix is 0-based, and thus row indices must be in the range 0 to $n - 1$. The first entry `Ap[0]` must be zero. The total number of entries in the matrix is thus $nz = Ap[n]$.

The matrix must be square, but it does not need to be symmetric. The `amd_order` routine constructs the nonzero pattern of $\mathbf{B} = \mathbf{A} + \mathbf{A}^T$ (without forming $\mathbf{A}^T$ explicitly), and then orders the matrix $\mathbf{B}$. Thus, either the lower triangular part of $\mathbf{A}$, the upper triangular part, or any combination may be passed. The transpose $\mathbf{A}^T$ may also be passed to `amd_order`. The diagonal entries may be present, but are ignored.

The input to `amd_order` must have sorted columns because it uses an in-place algorithm to construct $\mathbf{A} + \mathbf{A}^T$ without first constructing $\mathbf{A}^T$. This saves memory, but places an additional restriction on the input. If the input matrix has columns with unsorted and/or duplicate row indices, it is not valid as input to `amd_order`. To handle this case, the `amd_preprocess` routine is provided. It sorts, transposes, and removes duplicate entries from its input matrix, returning its result as another compressed-column matrix $\mathbf{R}$ which can then be passed to `amd_order`.

Control parameters are set an optional Control array. It is optional in the sense that if a NULL pointer is passed for the Control input argument, then default control parameters are used. The two control parameters are:

- (1) `Control[AMD_DENSE]` (or `Control(1)` in MATLAB). Controls the threshold for “dense” rows/columns. A dense row/column in $\mathbf{A} + \mathbf{A}^T$ can cause AMD to spend significant time in ordering the matrix. If `Control[AMD_DENSE]  $\geq 0$` , rows/columns with more than `Control[AMD_DENSE]  $\sqrt{n}$` entries are ignored during the ordering, and placed last in the output order. The default value

of `Control[AMD_DENSE]` is 10. If negative, no rows/columns are treated as “dense.” Rows/columns with 16 or fewer off-diagonal entries are never considered “dense.”

- (2) `Control[AMD_AGGRESSIVE]` (or `Control(2)` in MATLAB). Controls whether or not to use aggressive absorption, in which a prior element is absorbed into the current element if it is a subset of the current element, even if it is not adjacent to the current pivot element (refer to Amestoy et al. [1996] for more details). The default value is nonzero, which means that aggressive absorption will be performed. This nearly always leads to a better ordering (because the approximate degrees are more accurate) and a lower execution time. There are cases where it can lead to a slightly worse ordering, however. Set `Control[AMD_AGGRESSIVE] = 0` to turn it off.

Statistics are returned in the `Info` array (if `Info` is `NULL`, then no statistics are returned). Refer to `amd.h` file, for more details (14 different statistics are returned, so the list is not included here).

The following program illustrates the basic use of AMD:

```
#include <stdio.h>
#include "amd.h"

int n = 5 ;
int Ap [ ] = { 0, 2, 6, 10, 12, 14 } ;
int Ai [ ] = { 0,1, 0,1,2,4, 1,2,3,4, 2,3, 1,4 } ;
int P [5] ;

int main (void)
{
    int k ;
    (void) amd_order (n,Ap,Ai,P, (double *) NULL, (double *) NULL);
    for (k = 0 ; k < n ; k++) printf ("P [%d] = %d\n", k, P [k]);
    return (0) ;
}
```

The `Ap` and `Ai` arrays represent the binary matrix

$$\mathbf{A} = \begin{bmatrix} \mathbf{1} & 1 & 0 & 0 & 0 \\ \mathbf{1} & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & \mathbf{1} & 0 \\ 0 & 0 & 1 & \mathbf{1} & 0 \\ 0 & 1 & 1 & 0 & \mathbf{1} \end{bmatrix}.$$

AMD constructs the pattern of $\mathbf{A} + \mathbf{A}^T$, and returns a permutation vector of (0, 3, 1, 4, 2). More example programs are included with the AMD package.

4. USING AMD IN A FORTRAN PROGRAM

Two Fortran versions of AMD are provided. The AMD routine computes the approximate minimum degree ordering, using aggressive absorption. The AMDBAR

routine is identical, except that it does not perform aggressive absorption. The AMD routine is essentially identical to the HSL routine MC47B/BD [HSL 2002]. Note that earlier versions of the Fortran AMD and AMDBAR routines included an IOVFLO argument, which is no longer present.

In contrast to the C version, the Fortran routines require a symmetric nonzero pattern, with no diagonal entries present although the MC47A/AD wrapper in HSL allows duplicates, ignores out-of-range entries, and only uses entries from the upper triangular part of the matrix. Although we have an experimental Fortran code for treating “dense” rows, the Fortran codes in this release do not treat “dense” rows and columns of $\mathbf{A}$ differently, and thus their run time can be high if there are a few dense rows and columns in the matrix. They do not perform a post-ordering of the elimination tree, compute statistics on the ordering, or check the validity of their input arguments. These facilities are provided by MC47A/AD and other subroutines from HSL. Only one integer version of each Fortran routine is provided. Both Fortran routines overwrite the user’s input matrix, in contrast to the C version. The C version does not return the elimination or assembly tree. The Fortran version returns an assembly tree; refer to the User Guide for details. The following is the syntax of the AMD Fortran routine. The AMDBAR routine is identical except for the routine name.

```

      INTEGER N, IWLEN, PFREE, NCPMA, IW (IWLEN), PE (N), DEGREE (N), NV (N),
$      NEXT (N), LAST (N), HEAD (N), ELEN (N), W (N), LEN (N)
      CALL AMD (N, PE, IW, LEN, IWLEN, PFREE, NV, NEXT,
$      LAST, HEAD, ELEN, DEGREE, NCPMA, W)

```

The input matrix is provided to AMD and AMDBAR in three arrays, PE, of size N, LEN, of size N, and IW, of size IWLEN. The size of IW must be at least $NZ + N$. The recommended size is $1.2 \cdot NZ + N$. On input, the indices of nonzero entries in row I are stored in IW. PE(I) is the index in IW of the start of row I. LEN(I) is the number of entries in row I. The matrix is 1-based, with row and column indices in the range 1 to N. Row I is contained in IW (PE(I) $\cdots$ PE(I) + LEN(I) - 1). The diagonal entries must not be present. The indices within each row must not contain any duplicates, but they need not be sorted. The rows themselves need not be in any particular order, and there may be empty space between the rows. If LEN(I) is zero, then there are no off-diagonal entries in row I, and PE(I) is ignored. The integer PFREE defines what part of IW contains the user’s input matrix, which is held in IW(1 $\cdots$ PFREE - 1). The contents of IW and LEN are undefined on output, and PE is modified to contain information about the ordering.

As the algorithm proceeds, it modifies the IW array, placing the pattern of the partially eliminated matrix in IW(PFREE $\cdots$ IWLEN). If this space is exhausted, the space is compressed. The number of compressions performed on the IW array is returned in the scalar NCPMA. The value of PFREE on output is the length of IW required for no compressions to be needed.

The output permutation is returned in the array LAST, of size N. If $I = \text{LAST}(K)$, then I is the Kth row in the permuted matrix. The inverse permutation is returned in the array ELEN, where $K = \text{ELEN}(I)$ if I is the Kth row in the permuted matrix. On output, the PE and NV arrays hold the assembly tree (refer to the

User Guide for details). The other size N arrays (DEGREE, HEAD, NEXT, and W) are used as workspace, and are not defined on input or output.

The following program illustrates the basic use of the Fortran version of AMD. The AP and AI arrays represent the binary matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

in a conventional 1-based column-oriented form, except that the diagonal entries are not present. The matrix has the same as nonzero pattern of $\mathbf{A} + \mathbf{A}^T$ in the C program, in Section 3. The output permutation is (4, 1, 3, 5, 2). It differs from the permutation returned by the C routine `amd_order` because a post-order of the elimination tree has not yet been performed.

```

      INTEGER N, NZ, J, K, P, IWLEN, PFREE, NCPA
      PARAMETER (N = 5, NZ = 10, IWLEN = 17)
      INTEGER AP (N + 1), AI (NZ), LAST (N), PE (N), LEN (N), ELEN (N),
$      IW (IWLEN), DEGREE (N), NV (N), NEXT (N), HEAD (N), W (N)
      DATA AP / 1, 2,      5,      8,  9, 11/
      DATA AI / 2, 1,3,5, 2,4,5, 3,  2,3  /
C    load the matrix into the AMD workspace
      DO 10 J = 1,N
          PE (J) = AP (J)
          LEN (J) = AP (J + 1) - AP (J)
10    CONTINUE
      DO 20 P = 1,NZ
          IW (P) = AI (P)
20    CONTINUE
      PFREE = NZ + 1
C    order the matrix (destroys the copy of A in IW, PE, and LEN)
      CALL AMD (N, PE, IW, LEN, IWLEN, PFREE, NV, NEXT, LAST, HEAD,
$      ELEN, DEGREE, NCPA, W)
      DO 60 K = 1, N
          PRINT 50, K, LAST (K)
50      FORMAT ('P (',I2,',) = ', I2)
60    CONTINUE
      END
```

REFERENCES

- AMESTOY, P. R., DAVIS, T. A., AND DUFF, I. S. 1996. An approximate minimum degree ordering algorithm. *SIAM J. Matrix Anal. Applic.* 17, 4, 886–905.
- DAVIS, T. A., GILBERT, J. R., LARIMORE, S. I., AND NG, E. G. 2004. A column approximate minimum degree ordering algorithm. *ACM Trans. Math. Softw.* 30, 3 (Sept.), 000–000.
- GEORGE, A. AND LIU, J. W. H. 1989. The evolution of the minimum degree ordering algorithm. *SIAM Rev.* 31, 1, 1–19.

- HENDRICKSON, B. AND ROTHBERG, E. 1999. Improving the runtime and quality of nested dissection ordering. *SIAM J. Sci. Comput.* 20, 468–489.
- HSL. 2002. HSL 2002: A collection of Fortran codes for large scale scientific computation. <http://www.cse.clrc.ac.uk/nag/hsl>.
- KARYPIS, G. AND KUMAR, V. 1998. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM J. Sci. Comput.* 20, 359–392.
- PELLEGRINI, F., ROMAN, J., AND AMESTOY, P. 2000. Hybridizing nested dissection and halo approximate minimum degree for efficient sparse matrix ordering. *Concurrency: Practice and Experience* 12, 68–84.
- ROTHBERG, E. AND EISENSTAT, S. C. 1998. Node selection strategies for bottom-up sparse matrix orderings. *SIAM J. Matrix Anal. Applic.* 19, 3, 682–695.
- SCHULZ, J. 2001. Towards a tighter coupling of bottom-up and top-down sparse matrix ordering methods. *BIT* 41, 4, 800–841.

Received May 2003; revised February 2004; accepted April 2004